



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 05: Herencia y Polimorfismo

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

19 de Agosto, 2025

ECA

Habr  un beneficio por contestar la ECA

- 1 Si contestan la ECA todas las semanas ganar n una d cima a su promedio final
- 2 Esta d cima podr a ayudarlos a pasar
- 3 El  nico requisito es que $NE \geq 2.95$
- 4 Vamos a perdonar hasta 2 veces que se les olvide contestar

Cuestionario cap 3

- Las funciones en tu programa deben ser MUY cortas. De ser posible, 5 líneas o menos.

- Las funciones en tu programa deben ser MUY cortas. De ser posible, 5 líneas o menos.
- Generalmente, el código dentro de un if debería ser una sola línea (y esa línea debería ser un llamado a otra función).

- Las funciones en tu programa deben ser MUY cortas. De ser posible, 5 líneas o menos.
- Generalmente, el código dentro de un if debería ser una sola línea (y esa línea debería ser un llamado a otra función).
- No es deseable que el código de una función tenga más de dos niveles de indentación.

- Las funciones en tu programa deben ser MUY cortas. De ser posible, 5 líneas o menos.
- Generalmente, el código dentro de un if debería ser una sola línea (y esa línea debería ser un llamado a otra función).
- No es deseable que el código de una función tenga más de dos niveles de indentación.
- Se recomienda que toda función retorne un valor.

- Las funciones en tu programa deben ser MUY cortas. De ser posible, 5 líneas o menos.
- Generalmente, el código dentro de un if debería ser una sola línea (y esa línea debería ser un llamado a otra función).
- No es deseable que el código de una función tenga más de dos niveles de indentación.
- Se recomienda que toda función retorne un valor.
- Toda función debe hacer UNA sola cosa.

- Las funciones en tu programa deben ser MUY cortas. De ser posible, 5 líneas o menos.
- Generalmente, el código dentro de un if debería ser una sola línea (y esa línea debería ser un llamado a otra función).
- No es deseable que el código de una función tenga más de dos niveles de indentación.
- Se recomienda que toda función retorne un valor.
- Toda función debe hacer UNA sola cosa.
- Nunca hay que tener "switch statements" en tu código.

- Las funciones en tu programa deben ser MUY cortas. De ser posible, 5 líneas o menos.
- Generalmente, el código dentro de un if debería ser una sola línea (y esa línea debería ser un llamado a otra función).
- No es deseable que el código de una función tenga más de dos niveles de indentación.
- Se recomienda que toda función retorne un valor.
- Toda función debe hacer UNA sola cosa.
- Nunca hay que tener "switch statements" en tu código.
- El nombre de una función debe indicar claramente lo que hace. No importa que sea largo.

- Una forma de hacer tu código más fácil de entender es reducir el número de argumentos que reciben tus funciones.

Cuestionario cap 3

- Una forma de hacer tu código más fácil de entender es reducir el número de argumentos que reciben tus funciones.
- Hay que evitar crear funciones que reciban booleanos.

Cuestionario cap 3

- Una forma de hacer tu código más fácil de entender es reducir el número de argumentos que reciben tus funciones.
- Hay que evitar crear funciones que reciban booleanos.
- Los bloques dentro de una función deben tener una única entrada y una única salida. Por lo mismo, clean code considera que tener dos returns en una función se una mala práctica.

Cuestionario cap 3

- Una forma de hacer tu código más fácil de entender es reducir el número de argumentos que reciben tus funciones.
- Hay que evitar crear funciones que reciban booleanos.
- Los bloques dentro de una función deben tener una única entrada y una única salida. Por lo mismo, clean code considera que tener dos returns en una función se una mala práctica.
- Si algo falla en tu función lo mejor es retornar un código de error porque usar bloques de try y catch ensucian demasiado el código.

- Una forma de hacer tu código más fácil de entender es reducir el número de argumentos que reciben tus funciones.
- Hay que evitar crear funciones que reciban booleanos.
- Los bloques dentro de una función deben tener una única entrada y una única salida. Por lo mismo, clean code considera que tener dos returns en una función se una mala práctica.
- Si algo falla en tu función lo mejor es retornar un código de error porque usar bloques de try y catch ensucian demasiado el código.
- C# tiene la siguiente función estática: **int.TryParse(value, out number)**. Esta función intenta transformar el string value en un int. Si lo logra, retorna true. Si no lo logra, retorna false. Además, el valor transformado en int es guardado en number. Esta función es fácil de utilizar gracias a que sigue los principios discutidos en clean code.

Herencia

Existen tres tipos principales de relaciones entre clases:

- Composición.
- Asociación.
- Herencia.

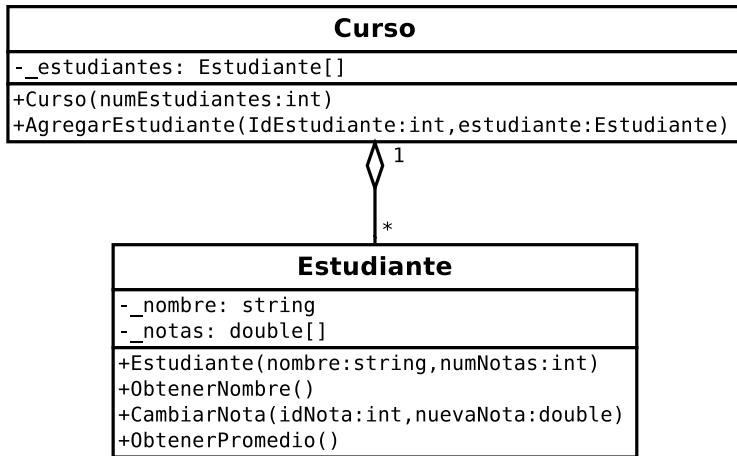
Composición

Composición: La clase A tiene como atributo un objeto de la clase B.

Composición: La clase A tiene como atributo un objeto de la clase B.

```
2 namespace HelloWorld;
3
4 public class Curso
5 {
6     private Estudiante[] _estudiantes;
7     public Curso(int numEstudiantes)
8     {
9         _estudiantes = new Estudiante[numEstudiantes];
10    }
11    public void AgregarEstudiante(int idEstudiante, Estudiante
estudiante)
12    {
13        _estudiantes[idEstudiante] = estudiante;
14    }
15 }
```

Composición: La clase A tiene como atributo un objeto de la clase B.

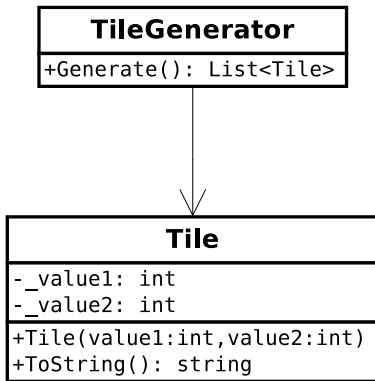


Asociación: La clase A usa un objeto de la clase B en alguno de sus métodos.

Asociación: La clase A usa un objeto de la clase B en alguno de sus métodos.

```
2 namespace Clase3;
3
4 public class TileGenerator
5 {
6     public List<Tile> Generate()
7     {
8         List<Tile> tiles = new List<Tile>();
9         for (int i = 0; i < 7; i++)
10         {
11             for (int j = i; j < 7; j++)
12                 tiles.Add(new Tile(i, j));
13         }
14
15         return tiles;
16     }
17 }
```

Asociación: La clase A usa un objeto de la clase B en alguno de sus métodos.



Herencia

Herencia: La clase A es una *especialización* de la clase B. **Sintaxis** "class A:B{...}"

Herencia

Herencia: La clase A es una *especialización* de la clase B. **Sintaxis** "class A:B{...}"

```
2 namespace Clase3;
3
4 public class Base
5 {
6     private int _a;
7     protected int B;
8
9     public Base(int a, int b)
10    {
11        _a = a;
12        B = b;
13    }
14
15    public int GetA() { return _a; }
16 }
```

Herencia

Herencia: La clase A es una *especialización* de la clase B. **Sintaxis** "class A:B{...}"

```
2 namespace Clase3;
3
4 public class Derived:Base
5 {
6     private string _s;
7
8     public Derived(string s, int a, int b)
9         : base(a, b)
10    {
11        _s = s;
12    }
13
14    public int GetB() { return B; }
15    public string GetS() { return _s; }
16 }
```

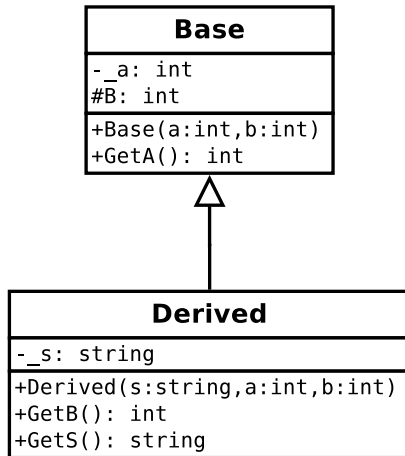
Herencia

Herencia: La clase A es una *especialización* de la clase B. **Sintaxis** "class A:B{...}"

```
2 using Clase3;  
3  
4 Derived d = new Derived("hola", 10, 5);  
5  
6 Console.WriteLine("A = " + d.GetA()); // A = 10  
7 Console.WriteLine("B = " + d.GetB()); // B = 5  
8 Console.WriteLine("S = " + d.GetS()); // S = hola
```

Herencia

Herencia: La clase A es una *especialización* de la clase B. **Sintaxis** "class A:B{...}"



Herencia: observaciones

1. Usamos `base(...)` en el constructor para llamar al constructor de la clase padre.

```
8   public Derived(string s, int a, int b)
9       : base(a, b)
10  {
11      _s = s;
12  }
```

Por defecto, se llama a `base()` → al constructor sin argumentos.

Herencia: observaciones

1. Usamos `base(...)` en el constructor para llamar al constructor de la clase padre.

```
8   public Derived(string s, int a, int b)
9       : base(a, b)
10  {
11      _s = s;
12  }
```

Por defecto, se llama a `base()` → al constructor sin argumentos.

2. La clase derivada solo puede acceder a los atributos/métodos que sean `public` o `protected` definidos en su clase base.

```
4   public class Base
5   {
6       private int _a;
7       protected int B;
```

Por ejemplo, desde `Derived:Base` podemos acceder a `B` pero no a `_a`.

3. Podemos asignar a variables del tipo Base objetos del tipo Derived.

```
2 Base d = new Derived("hola", 10, 5); // Funciona!
```


Herencia: observaciones

3. Podemos asignar a variables del tipo Base objetos del tipo Derived.

```
2 Base d = new Derived("hola", 10, 5); // Funciona!
```

4. Pero **no** podemos asignar a variables del tipo Derived objetos del tipo Base.

```
2 Derived d = new Base(10, 5); // ERROR!
```

Herencia: observaciones

3. Podemos asignar a variables del tipo Base objetos del tipo Derived.

```
2 Base d = new Derived("hola", 10, 5); // Funciona!
```

4. Pero **no** podemos asignar a variables del tipo Derived objetos del tipo Base.

```
2 Derived d = new Base(10, 5); // ERROR!
```

5. Finalmente, el tipo de la variable define los métodos y atributos que son visibles desde fuera (**no** el tipo del objeto).

```
2 Derived d = new Derived("hola", 10, 5); // Funciona!  
3 Console.WriteLine("A = " + d.GetA()); // Funciona!  
4 Console.WriteLine("B = " + d.GetB()); // Funciona!  
5 Console.WriteLine("S = " + d.GetS()); // Funciona!
```

Herencia: observaciones

3. Podemos asignar a variables del tipo Base objetos del tipo Derived.

```
2 Base d = new Derived("hola", 10, 5); // Funciona!
```

4. Pero **no** podemos asignar a variables del tipo Derived objetos del tipo Base.

```
2 Derived d = new Base(10, 5); // ERROR!
```

5. Finalmente, el tipo de la variable define los métodos y atributos que son visibles desde fuera (**no** el tipo del objeto).

```
2 Base d = new Derived("hola", 10, 5); // Funciona!  
3 Console.WriteLine("A = " + d.GetA()); // Funciona!  
4 Console.WriteLine("B = " + d.GetB()); // ERROR!  
5 Console.WriteLine("S = " + d.GetS()); // ERROR!
```

Sobreescritura de métodos

Herencia: sobreescritura de métodos (override)

Ahora bien, lo que hace poderosa a la herencia es la sobreescritura de métodos :)

Herencia: sobreescritura de métodos (override)

Ahora bien, lo que hace poderosa a la herencia es la sobreescritura de métodos :)

Para sobreescribir un método de la clase base se necesitan dos cosas:

- 1 El método debe ser declarado como `virtual` en la clase base.
- 2 El método debe ser marcado con el comando `override` en la clase derivada.

Al llamar al método se ejecutará la versión de la clase derivada.

Herencia: sobrescritura de métodos (override)

```
2 namespace Clase3;
3
4 public class Person
5 {
6     private string _name;
7
8     public Person(string name)
9     {
10         _name = name;
11     }
12
13     public virtual void Greet()
14     {
15         Console.WriteLine("Hola, me llamo " + _name);
16     }
17 }
```

Herencia: sobrescritura de métodos (override)

```
2 namespace Clase3;
3
4 public class Worker:Person
5 {
6     private string _company;
7     public Worker(string name, string company) : base(name)
8     {
9         _company = company;
10    }
11
12    public override void Greet()
13    {
14        base.Greet();
15        Console.WriteLine("\ty trabajo en: " + _company);
16    }
17 }
```


Herencia: sobrescritura de métodos (override)

```
2 using Clase3;  
3  
4 Person p1 = new Person("Rodrigo");  
5 Person p2 = new Worker("Guillermo", "UTEM");  
6 p1.Greet();  
7 p2.Greet();  
8 /* OUTPUT  
9 Hola, me llamo Rodrigo  
10 Hola, me llamo Guillermo  
11         y trabajo en: UTEM  
12 */
```

Observaciones

1. `p2.Greet()` usa la versión especializada de `Greet()` aunque `p2` sea un `Person`.
2. `base.Greet()` permite llamar a la versión del método `Greet()` en la clase base.

Herencia: sobrescritura de métodos (override)

Ahora agreguemos el método Work() a Worker.

```
4 public class Worker:Person {  
5     private string _company;  
6     public Worker(string name, string company) : base(name) {  
7         _company = company;  
8     }  
9     public override void Greet() {  
10         base.Greet();  
11         Console.WriteLine("\ty trabajo en: " + _company);  
12     }  
13     public void Work() { Console.WriteLine("A mirar Twitter :)"); }  
14 }
```

Herencia: sobrescritura de métodos (override)

Ahora agreguemos el método Work() a Worker.

```
4 public class Worker:Person {  
5     private string _company;  
6     public Worker(string name, string company) : base(name) {  
7         _company = company;  
8     }  
9     public override void Greet() {  
10         base.Greet();  
11         Console.WriteLine("\ty trabajo en: " + _company);  
12     }  
13     public void Work() { Console.WriteLine("A mirar Twitter :)"); }  
14 }
```

Si declaramos una variable de tipo Worker entonces podemos usar Work().

```
2 Worker w = new Worker("Guillermo", "UTEM");  
3 w.Work(); // Funciona!
```

Herencia: sobrescritura de métodos (override)

Ahora agreguemos el método Work() a Worker.

```
4 public class Worker:Person {  
5     private string _company;  
6     public Worker(string name, string company) : base(name) {  
7         _company = company;  
8     }  
9     public override void Greet() {  
10         base.Greet();  
11         Console.WriteLine("\ty trabajo en: " + _company);  
12     }  
13     public void Work() { Console.WriteLine("A mirar Twitter :)"); }  
14 }
```

Pero si la variable es de tipo Person entonces **no** podemos usar Work().

```
4 Person p = new Worker("Guillermo", "UTEM");  
5 p.Work(); // Error!
```

Ejemplo practico

Crea la clase `Figure` con los siguientes métodos:

- `Area()`: Retorna el área de la figura.
- `Perimeter()`: Retorna el perímetro de la figura.
- `Show()`: Muestra en consola el área y perímetro.

Ahora crea la clases `Rectangle:Figure` y `Circle:Figure`.

Finalmente, crea un arreglo con un círculo, un rectángulo y muestra su información usando el método `Show()`.

Herencia: sobreescritura de métodos (override)

```
1 public class Figure
2 {
3
4
5
6
7 }
```

Herencia: sobreescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area()
4     {
5
6     }
7 }
```

Herencia: sobreescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area()
4     {
5         // ACA SALE UN ERROR!
6     }
7 }
```


Herencia: sobrescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area()
4     {
5         return 0;
6     }
7 }
```

Herencia: sobreescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4
5
6
7 }
```

Herencia: sobreescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4     public virtual double Perimeter() => 0;
5
6
7 }
```

Herencia: sobrescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4     public virtual double Perimeter() => 0;
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

Herencia: sobrescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4     public virtual double Perimeter() => 0;
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

```
1 public class Circle
2 {
3
4
5
6
7
8
9 }
```

Herencia: sobrescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4     public virtual double Perimeter() => 0;
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

```
1 public class Circle : Figure
2 {
3
4
5
6
7
8
9 }
```

Herencia: sobrescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4     public virtual double Perimeter() => 0;
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

```
1 public class Circle : Figure
2 {
3     private double _radius;
4     public Circle(double radius) => _radius = radius;
5
6
7
8
9 }
```

Herencia: sobrescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4     public virtual double Perimeter() => 0;
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

```
1 public class Circle : Figure
2 {
3     private double _radius;
4     public Circle(double radius) => _radius = radius;
5     public double Area()
6         => return Math.PI * Math.Pow(_radius, 2);
7
8
9 }
```


Herencia: sobrescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4     public virtual double Perimeter() => 0;
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

```
1 public class Circle : Figure
2 {
3     private double _radius;
4     public Circle(double radius) => _radius = radius;
5     public double Area()
6         => return Math.PI * Math.Pow(_radius, 2);
7     public double Perimeter()
8         => return 2 * Math.PI * _radius;
9 }
```

Herencia: sobrescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4     public virtual double Perimeter() => 0;
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

```
1 Circle circle = new Circle(4);
2 circle.Show();
3 // Area: 50.3
4 // Perimetro: 25.1
```

Herencia: sobrescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4     public virtual double Perimeter() => 0;
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

```
1 Figure circle = new Circle(4);
2 circle.Show();
3 // Area: 0.0
4 // Perimetro: 0.0
```

Herencia: sobrescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4     public virtual double Perimeter() => 0;
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

```
1 public class Circle : Figure
2 {
3     private double _radius;
4     public Circle(double radius) => _radius = radius;
5     public double Area()
6         => return Math.PI * Math.Pow(_radius, 2);
7     public double Perimeter()
8         => return 2 * Math.PI * _radius;
9 }
```

Herencia: sobrescritura de métodos (override)

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4     public virtual double Perimeter() => 0;
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

```
1 public class Circle : Figure
2 {
3     private double _radius;
4     public Circle(double radius) => _radius = radius;
5     public override double Area()
6         => return Math.PI * Math.Pow(_radius, 2);
7     public override double Perimeter()
8         => return 2 * Math.PI * _radius;
9 }
```

Herencia: sobrescritura de métodos (override)

```
1 public class Rectangle:Figure {  
2     private double _width;  
3     private double _height;  
4     public Rectangle(double width, double height) {  
5         _width = width;  
6         _height = height;  
7     }  
8     public override double Area() => _width * _height;  
9     public override double Perimeter() => 2*(_width+_height);  
10 }
```

Herencia: sobrescritura de métodos (override)

```
1 public class Rectangle:Figure {  
2     private double _width;  
3     private double _height;  
4     public Rectangle(double width, double height) {  
5         _width = width;  
6         _height = height;  
7     }  
8     public override double Area() => _width * _height;  
9     public override double Perimeter() => 2*(_width+_height);  
10 }
```

```
1 Figure[] figures = new Figure[]{new Rectangle(4,2), new Circle(3.2)};  
2 foreach(Figure figure in figures)  
3     figure.Show();
```

Clases abstractas

Herencia: clases abstractas

Hacer una clase como la siguiente trae algunas complicaciones (¿por qué?:

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4     public virtual double Perimeter() => 0;
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

Herencia: clases abstractas

Hacer una clase como la siguiente trae algunas complicaciones (¿por qué?:

```
1 public class Figure
2 {
3     public virtual double Area() => 0;
4     public virtual double Perimeter() => 0;
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

Problemas:

- Queremos que el usuario pueda crear objetos de subtipos de Figure, como círculos y rectángulos. Pero el código actual le permite hacer: `new Figure()`.
- Hacer implementaciones dummies de `Area()` y `Perimeter()` no tiene mucho sentido
- Nos gustaría forzar que los subtipos de Figure sobrescriban los métodos `Area()` y `Perimeter()`.

Herencia: clases abstractas

Solución: definir Figure como una clase abstracta.

Herencia: clases abstractas

Solución: definir Figure como una clase abstracta.

Para definir una clase abstracta basta con marcarla como abstract.

```
1 public abstract class Figure
2 {
3     public abstract double Area();
4     public abstract double Perimeter();
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

Herencia: clases abstractas

Solución: definir Figure como una clase abstracta.

Para definir una clase abstracta basta con marcarla como abstract.

```
1 public abstract class Figure
2 {
3     public abstract double Area();
4     public abstract double Perimeter();
5     public void Show()
6         => Console.WriteLine($"Area: {Area()}\nPerimetro: {Perimeter()}");
7 }
```

Cuando una clase se declara abstracta NO es posible crear instancias de ella.

- "Figure f1 = new Rectangle(4, 3);" → Funciona!
- "Figure f2 = new Circle(1.2);" → Funciona!
- "Figure f3 = new Figure();" → Falla! :)

Herencia: clases abstractas

Las clases abstractas también nos permiten definir métodos abstractos.

```
3 public abstract double Area();  
4 public abstract double Perimeter();
```

Herencia: clases abstractas

Las clases abstractas también nos permiten definir métodos abstractos.

```
3 public abstract double Area();  
4 public abstract double Perimeter();
```

Observaciones:

- Los métodos abstractos definen la firma del método (su nombre, parámetros y tipo de retorno) pero **no** su implementación.
- Todo método abstracto **debe** ser sobrescrito por las clases que hereden de la clase abstracta (a menos que la clase derivada también sea abstracta).
- Nota que los métodos abstractos son intrínsecamente `virtual`.
- Sobrescribir métodos abstractos es igual que sobrescribir métodos `virtual`.

Herencia: clases abstractas

Las clases abstractas también nos permiten definir métodos abstractos.

```
3 public abstract double Area();  
4 public abstract double Perimeter();
```

Observaciones:

- Los métodos abstractos definen la firma del método (su nombre, parámetros y tipo de retorno) pero **no** su implementación.
- Todo método abstracto **debe** ser sobrescrito por las clases que hereden de la clase abstracta (a menos que la clase derivada también sea abstracta).
- Nota que los métodos abstractos son intrínsecamente `virtual`.
- Sobreescribir métodos abstractos es igual que sobrescribir métodos `virtual`.

De hecho los códigos de `Circle` y `Rectangle` quedan igual que antes.


```
1 public class Circle : Figure
2 {
3     private double _radius;
4     public Circle(double radius) => _radius = radius;
5     public override double Area()
6         => return Math.PI * Math.Pow(_radius, 2);
7     public override double Perimeter()
8         => return 2 * Math.PI * _radius;
9 }
```

```
1 public class Rectangle:Figure {
2     private double _width;
3     private double _height;
4     public Rectangle(double width, double height) {
5         _width = width;
6         _height = height;
7     }
8     public override double Area() => _width * _height;
9     public override double Perimeter() => 2*(_width+_height);
10 }
```

Interfaces

En C#, las *interfaces* nos permiten explotar el polimorfismo en todo su esplendor.

Interfaces

En C#, las *interfaces* nos permiten explotar el polimorfismo en todo su esplendor.

Las interfaces son listas de métodos a ser implementados:

```
2 interface ILivingBeing {  
3     void Bear(); // Nacer  
4     void Grow(); // Crecer  
5     ILivingBeing Mate(ILivingBeing couple); // Reprod.  
6     void Die(); // Morir  
7 }
```

... toda clase puede implementar cuántas interfaces quiera.

La sintaxis para implementar una interfaz es igual que para heredar:

- `class <NombreClase>:<NombreInterfaz>{...}`

La sintaxis para implementar una interfaz es igual que para heredar:

- `class <NombreClase>:<NombreInterfaz>{...}`

Pero se pueden implementar varias interfaces (y heredar de una clase).

- `class <NombreClase>:<NombreInterfaz1>,<NombreInterfaz2>,... {...}`

- `class <NombreClase>:<NombreClaseBase>,<NombreInterfaz1>,... {...}`

La sintaxis para implementar una interfaz es igual que para heredar:

- `class <NombreClase>:<NombreInterfaz>{...}`

Pero se pueden implementar varias interfaces (y heredar de una clase).

- `class <NombreClase>:<NombreInterfaz1>,<NombreInterfaz2>,... {...}`

- `class <NombreClase>:<NombreClaseBase>,<NombreInterfaz1>,... {...}`

Para implementar la interfaz tu clase debe implementar todos los métodos definidos en la interfaz (al igual que con los `abstract methods`). Sin embargo, no hay que poner `override`. Basta con definir el método no más.

Interfaces

```
2 class Dog : ILivingBeing{
3     private int _age;
4     public Dog(Dog father, Dog mother) { /*...*/ }
5     public void Bear(){
6         _age = 0;
7         Bark();
8     }
9     public void Grow() {
10         _age++;
11         GrowTeeth();
12         DevelopKillerInstinct();
13     }
14     public ILivingBeing Mate(ILivingBeing couple) {
15         return new Dog(this, (Dog)couple);
16     }
17     public void Die(){ Console.WriteLine("Arf!"); }
18     //... el resto del código no es relevante
19 }
```


Interfaces

Ahora podemos declarar variables del tipo de la interfaz y asignarle cualquier objeto que la implemente:

```
1 ILivingBeing dog = new dog(father, mother);  
2 dog.Bear();  
3 dog.Grow();  
4 ILivingBeing Doggie = dog.Mate(partner);  
5 dog.Die();
```

Interfaces

Ahora podemos declarar variables del tipo de la interfaz y asignarle cualquier objeto que la implemente:

```
1 ILivingBeing dog = new dog(father, mother);  
2 dog.Bear();  
3 dog.Grow();  
4 ILivingBeing Doggie = dog.Mate(partner);  
5 dog.Die();
```

Es decir, podemos definir métodos muy generales gracias a las interfaces:

```
2 void KillEveryone(ILivingBeing[] beings) {  
3     for(int i = 0; i < beings.Length; i++)  
4         beings[i].Die();  
5 }
```

... este método sirve para matar a cualquier que implemente ILivingBeing.

Interfaces

```
2 class Flower:ILivingBeing
3 {
4     private int _age;
5     public Flower(Flower father, Flower mother) { /*...*/ }
6     public void Bear() {
7         _age = 0;
8         CreatePetals();
9     }
10    public void Grow(){
11        _age++;
12        GrowRoots();
13    }
14    public ILivingBeing Mate(ILivingBeing couple){
15        return new Flower(this, (Flower)couple);
16    }
17    public void Die(){ Wither(); }
18    //... el resto del código no es relevante
19 }
```

... y nuestro código para matarlos a todos sigue funcionando:

```
2 void KillEveryone(ILivingBeing[] beings) {  
3     for(int i = 0; i < beings.Length; i++)  
4         beings[i].Die();  
5 }
```

... y nuestro código para matarlos a todos sigue funcionando:

```
2 void KillEveryone(ILivingBeing[] beings) {  
3     for(int i = 0; i < beings.Length; i++)  
4         beings[i].Die();  
5 }
```

Una interfaz es como una clase abstracta con puros métodos abstractos... pero:

- Se pueden implementar múltiples interfaces.
- No se necesita escribir ni `virtual` ni `override`.
- En general, implementar una interfaz es más eficiente que heredar.